

Подготовка к экзамену по C++ (ИТМО)

Тема 10: Исключения, Stack Unwinding и noexcept

Hardcore Theory Edition

18 июня 2026 г.

1 Основы исключений

Механизм исключений (`try / catch / throw`) позволяет передать информацию об ошибке туда, где знают, как её обработать. В C++ можно "бросить" (`throw`) вообще всё что угодно: число `throw 5;`, строку `throw "Error";`. Но по стандарту хорошего тона нужно бросать объекты-наследники класса `std::exception` (например, `std::runtime_error` или `std::out_of_range`).

2 Stack Unwinding (Раскрутка стека)

Когда происходит `throw`, программа немедленно прерывает выполнение текущей функции и начинает искать ближайший `catch`. Процесс "сворачивания" функций и возврата по цепочке вызовов назад называется **Раскруткой стека**. В этот момент компилятор **гарантированно вызывает деструкторы** для всех локальных объектов, созданных на стеке в покинутых функциях. Именно поэтому паттерн RAII и умные указатели работают безотказно.

3 Три Смертных Греха Исключений

3.1 Грех 1: Ловля по значению (Object Slicing)

Правило: Бросай по значению, лови по константной ссылке! `catch (const std::exception& e)`. Если вы поймаете исключение по значению: `catch (std::exception e)`, произойдет **Срезка (Slicing)**. Полиморфизм сломается. Вы скопируете только "базовую" часть исключения, а вся полезная информация из наследника (`std::runtime_error`) будет безвозвратно обрезана, и виртуальный метод `e.what()` вернет дефолтную строку базового класса.

3.2 Грех 2: Исключение из Деструктора

Правило: Деструктор **никогда** не должен выпускать исключения наружу! Если во время "Раскрутки стека" (поиска блока `catch`) деструктор какого-то локального объекта тоже выбросит исключение, в программе окажется **два активных исключения одновременно**. C++ не знает, какое из них обрабатывать, впадает в панику и мгновенно вызывает `std::terminate()` (Жесткий краш программы без возможности перехвата).

3.3 Грех 3: Порядок блоков catch

Если у вас несколько блоков `catch`, ловить нужно **от наследников к базовому классу**. Если вы напишете `catch (const std::exception& e)` первым, он поймает ВСЕ исключения, и блоки с `std::runtime_error` под ним никогда не выполнятся.

Суть на пальцах (Жизненная аналогия)

Ловля исключений — это рыбалка с несколькими сетями. Вы ставите сети на реке. Сначала нужно ставить сетку с крупными ячейками на акул (`std::runtime_error`). А уже ниже по течению — мелкую сеть на любую рыбу (`std::exception`). Если вы поставите мелкую сеть первой, акула запутается в ней, и вторая сеть останется пустой.

4 Ключевое слово `noexcept`

`noexcept` — это ваше обещание компилятору: *"Эта функция клянется никогда не бросать исключений"*.

- Все деструкторы в C++11 и выше по умолчанию являются `noexcept`.
- **Магия `std::vector`:** Когда вектор делает реаллокацию (расширение памяти), он должен перенести элементы в новый кусок памяти. Если у ваших элементов **Move-конструктор** помечен как `noexcept`, вектор быстро *переместит* их. Если вы **забыли** написать `noexcept`, вектор испугается: "А вдруг при перемещении вылетит исключение? Я не смогу откатить изменения!". В целях безопасности вектор откажется от перемещения и будет медленно **копировать** каждый элемент по одному!